Project 1 Report

Design

이번 과제는 xv6의 CPU 스케줄링 방식을 Round Robin에서 Complete Fair Scheduler (CFS)로 만드는 과제다. 명세에 따르면:

- 각 프로세스는 virtual runtime을 가진다. 이 virtual runtime이 낮은 프로세스부터 스케줄링 해준다.
- Nice라는 정수값이 있는데, 이 nice 값이 높으면 높을수록 스케줄링 시 적은 빈도로 선택된다.
- RUNNABLE 상태의 프로세스는 모두 Red-Black Tree에서 관리한다.

Lab 6에서 queue를 이용하여 RUNNABLE 프로세스를 관리했던 매커니즘을 그대로 사용하면 되겠다고 생각했다. `proc.c`의 관련 함수를 수정하거나 필요한 기능이 있으면 함수를 추가하고, Red-Black Tree 파일(`rbtree.c`, `rbtree.h`)의 구현 방식을 보고 필요하다면 이 부분을 수정하여 과제 명세의 요구 사항을 맞추고자 하였다.

Implementation

먼저 `proc.c`부터 일부 수정하였다.

```
13
14    struct rb_tree runnable_tree;
```

RUNNABLE 프로세스를 관리하는 Red-Black Tree 구조체 `runnable_tree`를 생성하였다. 이제 이 tree에 RUNNABLE 프로세스를 추가하고, 이 tree로부터 virtual runtime이 가장 작은 프로세스를 우선적으로 스케줄링 해주면 된다.

`proc.h`에 구현된 `proc` 구조체에도 수정이 필요하다.

```
108     int nice;
109     struct rb_node runnable_node;
110     uint64 vruntime;
```

과제 명세에 따라 프로세스마다 nice 값과 virtual runtime을 가져야 하고, Red-Black Tree에 삽입할 때 필요한 `rb_node`도 추가하였다.

잠시 `rbtree.h`로 넘어가서 Red-Black Tree의 구조부터 내가 원하는 대로 고쳤다.

```
15    struct rb_tree {
16      struct rb_node *root;
17      struct spinlock lock;
18    };
```

`rb_tree` 구조체에 `lock`을 만들었다. Tree 내의 공유 변수 동기화를 하기 위함이다.

```
20    void rb_insert(struct rb_tree *tree, struct rb_node *node);
21    void rb_delete(struct rb_tree *tree, struct rb_node *node);
```

`rb_insert` 함수는 `tree`와 `node`를 입력받아서 `tree`에 `node`를 넣어준다. 뭔가 복잡하기도 하고, `rbtree.c`에서 lock 기능을 구현하는 것보다 `proc.c`에 그냥 새로운 함수를 만드는 게 더 편할 것 같아서 `proc.c`에 `insert_process_into_runnable_tree` 함수를 만들었다.

```
void insert_process_into_runnable_tree(struct proc *p)
{
  acquire(&runnable_tree.lock);
  p->runnable_node.key = p->vruntime;
  p->runnable_node.data = p;
  rb_insert(&runnable_tree, &p->runnable_node);
  release(&runnable_tree.lock);
}
```

이 함수는 `proc` 구조체를 입력으로 받아 해당 프로세스를 `runnable_tree`에 삽입하는데, 이 때 `runnable_tree` 내부 공유 변수 동기화를 위해 `lock`을 먼저 `acquire`하고, `key`와 `data`에

각각 virtual runtime과 프로세스 구조체를 할당한다. 그리고 `rb_insert` 함수로 삽입을 실행한다. 모든 작업이 끝나면 `release`로 `lock`을 반납한다.

이후 `proc.c`를 본격적으로 수정하였다.

```
uint64 get_min_vruntime(void)
{
  uint64 min_vruntime = 0;
  int found = 0;

  acquire(&runnable_tree.lock);
  struct rb_node *min_node = rb_first(runnable_tree.root);
  if (min_node) {
    min_vruntime = min_node->key;
    release(&runnable_tree.lock);
    return min_vruntime;
  }
  release(&runnable_tree.lock);

  struct proc *p;
  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->state == RUNNING) {
      if(!found) {
        min_vruntime = p->vruntime;
        found = 1;
      }
      else if (p->vruntime < min_vruntime) min_vruntime = p
->vruntime;
    }
    release(&p->lock);
  }
  if(found) return min_vruntime;

  return 0;
}
```

`insert_process_into_runnable_tree` 함수와 마찬가지로 구현 편의를 위해 최소 virtual runtime을 return하는 `get_min_vruntime` 함수를 구현하였다.

1. RUNNABLE 프로세스가 존재하는 경우
→ 명세대로 RUNNABLE 프로세스들 중 최소 virtual runtime으로 갱신

2. RUNNABLE 프로세스는 없지만, RUNNING 중인 프로세스가 하나 이상 존재하는 경우
→ RUNNING 프로세스들의 virtual runtime 중 최솟값으로 갱신

3. RUNNABLE도 RUNNING도 존재하지 않는 경우
→ virtual runtime을 0으로 설정

1. RUNNABLE 프로세스가 존재하는 경우

```
acquire(&runnable_tree.lock);
struct rb_node *min_node = rb_first(runnable_tree.root);
if (min_node) {
        min_vruntime = min_node->key;
    release(&runnable_tree.lock);
    return min_vruntime;
}
release(&runnable_tree.lock);
```

Red-Black Tree에서의 최솟값을 반환하는 `rb_first` 함수에 `runnable_tree.root`를 넣고, 이 값을 `rb_node` 구조체 포인터 `min_node`에 넣었다. 만약 `runnable_tree`에 RUNNABLE 프로세스가 하나라도 있으면, 최소 virtual runtime을 구할 수 있기 때문에 `min_node`의 `key` 값을 미리 만들어둔 `min_vruntime`에 담고 이 값을 반환하여 함수를 종료한다.

2. RUNNABLE 프로세스는 없지만, RUNNING 중인 프로세스가 하나 이상 존재하는 경우

```
struct proc *p;
  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if (p->state == RUNNING) {
      if(!found) {
        min_vruntime = p->vruntime;
        found = 1;
      }
```

```
      else if (p->vruntime < min_vruntime) min_vruntime
= p->vruntime;
    }
    release(&p->lock);
  }
  if(found) return min_vruntime;
```

RUNNABLE 프로세스는 모두 `runnable_tree`에 있는데 만약 위에서 return이 되지 않았다면 RUNNABLE 프로세스가 없다는 소리다. 그러면 이제 RUNNING 프로세스를 찾아야 한다.

`int found`는 비록 int형 변수지만, 여기서는 boolean 변수처럼 0 또는 1 값을 가진다. 0이면 RUNNING 프로세스가 없다는 뜻이고, 1이면 있다는 뜻이다.

- RUNNING 프로세스가 정확히 1개인 경우 `if(!found)`문을 거쳐 `min_vruntime` 값을 `p`의 `vruntime`으로 설정하고 끝난다.
- RUNNING 프로세스가 여러 개인 경우, 제일 먼저 탐색된 RUNNING 프로세스는 `if(!found)`문을 거쳐 `found` 값을 1로 설정하기 때문에 더이상 `if(!found)`문은 사용되지 않는다. 그 후에 탐색된 RUNNING 프로세스는 기존의 `min_vruntime`보다 `vruntime`이 작다면 업데이트해주고, 그렇지 않으면 기존의 값을 그대로 사용한다.

`if(found)`문을 통해 RUNNING 프로세스 존재 여부를 확인하고, 그 RUNNING 프로세스 중 최소 `vruntime`을 반환해준다.

3. RUNNABLE도 RUNNING도 존재하지 않는 경우

```
return 0;
```

앞서 설정한 분기를 모두 통과하여 반환하지 않고 여기까지 왔다면 RUNNABLE도 RUNNING도 존재하지 않는 경우이므로 0을 반환해야 한다.

필요한 기능을 다 구현했으니 본격적으로 `proc.c`를 탐색하며 수정할 부분을 찾았다. Lab 6에서 Ctrl + F로 RUNNABLE이 나오는 부분을 찾아 그 부분에 queue의 삽입 함수를 추가했던 방식을 채용하였다.

```
274   p->state = RUNNABLE;
275   insert_process_into_runnable_tree(p);
```

`userinit()` 함수에 `insert_process_into_runnable_tree` 추가

```
350     np->state = RUNNABLE;
351     insert_process_into_runnable_tree(p: np);
```

`kfork()` 함수에 `insert_process_into_runnable_tree` 추가: child process를 추가해야 한다.

```
556     p->state = RUNNABLE;
557     insert_process_into_runnable_tree(p);
```

`yield()` 함수에 `insert_process_into_runnable_tree` 추가

```
643         p->state = RUNNABLE;
644         insert_process_into_runnable_tree(p);
```

`wakeup()` 함수에 `insert_process_into_runnable_tree` 추가

```
667         p->state = RUNNABLE;
668         insert_process_into_runnable_tree(p);
```

`kkill()` 함수에 `insert_process_into_runnable_tree` 추가

`scheduler()` 함수를 수정하여 CPU 스케줄링 방식을 Round Robin에서 CFS로 바꿔줬다.

```
  void
  scheduler(void)
  {
          ...
      acquire(&runnable_tree.lock);
      struct rb_node *min_node = rb_first(runnable_tree.roo
t);
      if (min_node) {
        rb_delete(&runnable_tree, min_node);
        p = (struct proc *)min_node->data;
        release(&runnable_tree.lock);

        acquire(&p->lock);
        if(p->state == RUNNABLE) {
          p->state = RUNNING;
          c->proc = p;

          int vruntime = 1;
          if(p->nice == -3) vruntime = 1;
          else if(p->nice == -2) vruntime = 2;
```

```
        else if(p->nice == -1) vruntime = 4;
        else if(p->nice == 0) vruntime = 8;
        else if(p->nice == 1) vruntime = 16;
        else if(p->nice == 2) vruntime = 32;
        p->vruntime += vruntime;

        swtch(&c->context, &p->context);
        c->proc = 0;
      }
      release(&p->lock);
    } else {
      release(&runnable_tree.lock);
      // nothing to run; stop running on this core until an
interrupt.
      asm volatile("wfi");
    }
  }
}
```

먼저 `rb_node` 구조체 포인터 `min_node`에 `vruntime`이 최솟값인 프로세스를 저장하고, 이 프로세스는 이제 CPU 스케줄러에 의해 RUNNING 상태로 바뀔 것이기 때문에 `rb_delete()` 함수를 이용하여 `runnable_tree`에서 제거해주었다.

이제 이 프로세스를 `p`에 넣고, `p`가 RUNNABLE 상태인 경우 RUNNING 상태로 바꿔주었다. 이때 프로세스 `p`에 대한 다른 CPU의 접근(멀티코어 환경을 가정)을 막기 위해 `acquire(&p->lock)`을 넣어주었다. 그리고 명세에 따라 `nice`가 1 증가할 때마다 `vruntime`이 2배 증가하도록 하였는데, `vruntime`의 구체적인 값에 대한 명세가 없어서 임의로 `nice`가 -3일 때 `vruntime`의 최솟값을 1로 기준을 잡았다.

`swtch()` 함수로 context switch를 실행하고 `release(&p->lock)`으로 프로세스 `p`에 대한 lock을 반납한다.

`proc.c`에서 `insert_process_into_runnable_tree` 함수 외에도 수정해줘야 할 부분들이 있었다.

```
96    initlock(&runnable_tree.lock, "runnable_tree_lock");
97    runnable_tree.root = 0;
```

먼저 `procinit()` 함수에서 `runnable_tree`의 `lock`과 `root` 값을 초기화해주었다.

```
168   found:
169     p->pid = allocpid();
170     p->state = USED;
171     p->nice = 0;
172     p->vruntime = 0;
```

`allocproc()` 함수에서 프로세스 `p`를 초기화하는 과정에 `nice`와 `vruntime`도 초기화해주었다.

```
340     np->vruntime = p->vruntime;
341     np->nice = p->nice;
```

`kfork()` 함수에서 자식 프로세스가 부모 프로세스의 `vruntime`과 `nice` 값을 승계하도록 설정해주었다.

```
632   void
633   wakeup(void *chan)
634   {
635     struct proc *p;
636     uint64 min_vruntime = get_min_vruntime();
637
638     for(p = proc; p < &proc[NPROC]; p++) {
639       if(p != myproc()){
640         acquire(&p->lock);
641         if(p->state == SLEEPING && p->chan == chan) {
642           p->vruntime = min_vruntime;
643           p->state = RUNNABLE;
644           insert_process_into_runnable_tree(p);
```

```
654   int
655   kkill(int pid)
656   {
657     struct proc *p;
658
659     uint64 min_vruntime = get_min_vruntime();
660     for(p = proc; p < &proc[NPROC]; p++){
661       acquire(&p->lock);
662       if(p->pid == pid){
663         p->killed = 1;
664         if(p->state == SLEEPING){
665           // Wake process from sleep().
666           p->vruntime = min_vruntime;
667           p->state = RUNNABLE;
668           insert_process_into_runnable_tree(p);
669         }
```

`wakeup()`과 `kkill()` 함수에서 공통적으로 프로세스를 깨워줘야 하는데, 이때 `get_min_vruntime()` 함수에서 구현한 로직에 따라 현재 RUNNABLE (없다면 RUNNING) 프로세스 중 `vruntime`의 최솟값으로 설정해줘야 하기 때문에 코드를 추가하였다.

```
int
set_nice(int nice)
{
  struct proc *p = myproc();
  if (p->state != RUNNING || nice < -3 || nice > 2)
    return 1;
  p->nice = nice;
  return 0;
}
```

마지막으로 `set_nice` system call 함수를 구현하였다. 현재 RUNNING 상태이면서 `nice` 값이 valid한 프로세스를 대상으로 해당 프로세스의 `nice` 값을 입력받은 `nice` 값으로 설정해주고 0을 반환하고, 그렇지 않은 경우 1을 반환한다.

`defs.h`, `sysproc.c`, `syscall.c`, `syscall.h`에 `set_nice` system call을 등록하고 user 단의 `user.h`, `usys.pl`에 추가하여 user program을 통해 호출할 수 있도록 만들었다.

```
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
```

```c
#define PNUM 6
int nice_vals[PNUM] = {-3,-2, -1, 0, 1, 2};

#define LOOP_COUNT 300000000

int main(int argc, char *argv[]) {
  printf("Starting CFS Test...\n");

  for (int i = 0; i < PNUM; i++) {
    int pid = fork();

    if (pid < 0) {
      printf("fork failed\n");
      exit(1);
    }

    if (pid == 0) {
      if (set_nice(nice_vals[i]) != 0) {
        printf("set_nice failed for nice %d\n", nice_vals
[i]);
        exit(1);
      }

      int start_tick = uptime();

      for (volatile int j = 0; j < LOOP_COUNT; j++) {

      }

      int end_tick = uptime();

      printf("PID: %d, Nice: %d, Ticks: %d\n", getpid(), ni
ce_vals[i], end_tick - start_tick);

      exit(0);
    }
  }
```

```
  for (int i = 0; i < PNUM; i++) {
    wait(0);
  }

  printf("CFS Test Completed.\n");
  exit(0);
}
```

User program `cfs.c`를 만들어서 CFS 스케줄링이 제대로 일어나는지 확인해봤다. Nice 값을 -3부터 2까지 총 6개의 프로세스에 할당하고, 각 프로세스가 종료할 때 자신의 PID, Nice, 그리고 연산에 소요된 시간을 출력하게 했다.

Results

```
/project1-yjkim0306-1  main ±  make qemu CPUS=1
```

먼저 `make qemu CPUS=1` 명령어로 xv6를 실행하였다.

```
xv6 kernel is booting

init: starting sh
$ cfs
Starting CFS Test...
PID: 4, Nice: -3, Ticks: 22
PID: 5, Nice: -2, Ticks: 26
PID: 6, Nice: -1, Ticks: 30
PID: 7, Nice: 0, Ticks: 35
PID: 8, Nice: 1, Ticks: 39
PID: 9, Nice: 2, Ticks: 41
CFS Test Completed.
$
```

cfs 테스트 결과 nice 값이 낮은 프로세스부터 먼저 종료되었고, 소요된 시간(CPU를 잡고 있던 시간)도 가장 낮은 프로세스부터 먼저 종료되었다. CFS 스케줄링이 제대로 구현된 것을 확인할 수 있었다.

GitHub Discussions 탭에 올라온 다른 테스트코드도 돌려보았다.

```
void work(int target_ticks, char who) {
  int start = uptime();
```

```
  printf("%c", who);
  while (uptime() - start < target_ticks) {}
}

void print_header(char *title) {
  printf("\n==================================================
====================\n");
  printf(" TEST: %s\n", title);
  printf("==================================================
==================\n");
}

int cfstest(char* s) {
  int pid1, pid2;

  printf("\n[CFS Scheduler Test]\n");

  // ---------------------------------------------------------
---
  // 1. set_nice() 리턴값 검증
  // ---------------------------------------------------------
---
  print_header("1. Validate return value of set_nice system
call");
  printf("[EXPECT] allows [-3, 2] and returns 1 on fail.\n
\n");

  int test_vals[]  = {-4, -3, -2, -1, 0, 1, 2, 3};
  int expect_ret[] = {1, 0, 0, 0, 0, 0, 0, 1};

  for(int i = 0; i < sizeof(test_vals)/sizeof(int); i++) {
    int res = set_nice(test_vals[i]);
    printf("set_nice(%d) -> Result: %d (Expected: %d) : ",
test_vals[i], res, expect_ret[i]);
    if (res == expect_ret[i]) printf("PASS\n");
    else { printf("FAIL\n"); exit(1); }
  }
  set_nice(0);
```

```
  // ------------------------------------------------------
---
  // 2. Wakeup할 때 minimum vruntime으로 설정하는지 테스트
  // ------------------------------------------------------
---
  print_header("2. Wakeup Test (CPUS=1)");
  printf("[EXPECT] After child wakes up, P and C should int
erleave evenly.\n\n");
  pid1 = fork();
  if (pid1 == 0) {
    pause(15); // pause child
    for(int i = 0; i < 15; i++) work(1, 'C');
    exit(0);
  } else {
    for(int i = 0; i < 30; i++) work(1, 'P');
    wait(0);
  }
  printf("\n\n");
  set_nice(0);

  // ------------------------------------------------------
---
  // 3. Fork 시 Nice & Vruntime 상속 검증
  // ------------------------------------------------------
---
  print_header("3. Nice value Fork Inheritance (CPUS=1)");
  printf("[EXPECT] P and C should interleave evenly (e.g.,
PPCPCPCP...).\n\n");

  set_nice(-2);
  work(5, 'P'); // 부모가 5틱 동안 먼저 일해서 vruntime 증가

  pid1 = fork();
  if (pid1 == 0) {
    for(int i = 0; i < 20; i++) work(1, 'C');
    exit(0);
```

```
  } else {
    for(int i = 0; i < 20; i++) work(1, 'P');
    wait(0);
  }
  printf("\n\n");
  set_nice(0);

  // ------------------------------------------------------
---
  // 4. Nice 값대로 vruntime이 증가하는지 검증
  // ------------------------------------------------------
---
  print_header("4. Nice value CPU Share (CPUS=1)");
  printf("[EXPECT] H 16 times, L 1 time, H 16 times ...\n
\n");

  pid1 = fork();
  if (pid1 == 0) {
    set_nice(2);
    for(int i = 0; i < 10; i++) work(1, 'L');
    exit(0);
  }

  pid2 = fork();
  if (pid2 == 0) {
    set_nice(-2);
    for(int i = 0; i < 60; i++) work(1, 'H');
    exit(0);
  }

  wait(0); wait(0);
  printf("\n\n");
  set_nice(0);

  // ------------------------------------------------------
---
  // 5. 멀티프로세서 데드락 발생 여부 검증
  // ------------------------------------------------------
```

```
---
  print_header("5. Check for deadlocks in Multiprocessor en
vironment");
  printf("[EXPECT] No deadlocks occur & Process ends in A,
B, C, D, E, F order or similar.\n\n");

  int pids[6];

  int nice_vals[6] = {-3, -2, -1, 0, 1, 2};

  char labels[6] = {'a', 'b', 'c', 'd', 'e', 'f'};
  printf("[a: -3] [b: -2] [c: -1] [d: 0] [e: 1] [f: 2]\n
\n");

  for(int i = 0; i < 6; i++) {
      pids[i] = fork();

      if (pids[i] == 0) {
          if (set_nice(nice_vals[i]) != 0) {
              printf("\n[!] Fail to set nice for %c\n", lab
els[i]);
              exit(0);
          }

          for(int j = 0; j < 100; j++) {
            if (j == 99) {
              printf("%c", labels[i] + 'A' - 'a');
            } else {
              work(1, labels[i]);
            }
          }

          exit(0);
      }
  }

  for(int i = 0; i < 6; i++) {
      wait(0);
```

```
    }
    printf("\n\n");

    printf("=====================================================
\n");
    printf(" ALL TESTS COMPLETED.\n");
    printf("=====================================================
\n");

    return 0;
}
```

qndks11님의 테스트 코드 실행 결과:

```
$ usertests cfstest
usertests starting
test cfstest:
[CFS Scheduler Test]

=====================================================================
 TEST: 1. Validate return value of set_nice system call
=====================================================================
[EXPECT] allows [-3, 2] and returns 1 on fail.

set_nice(-4) -> Result: 1 (Expected: 1) : PASS
set_nice(-3) -> Result: 0 (Expected: 0) : PASS
set_nice(-2) -> Result: 0 (Expected: 0) : PASS
set_nice(-1) -> Result: 0 (Expected: 0) : PASS
set_nice(0) -> Result: 0 (Expected: 0) : PASS
set_nice(1) -> Result: 0 (Expected: 0) : PASS
set_nice(2) -> Result: 0 (Expected: 0) : PASS
set_nice(3) -> Result: 1 (Expected: 1) : PASS

=====================================================================
 TEST: 2. Wakeup Test (CPUS=1)
=====================================================================
[EXPECT] After child wakes up, P and C should interleave evenly.

PPPPPPPPPPPPPPPPCPCPCPCPCPCPCPCPCPCPCPCPCPC


=====================================================================
 TEST: 3. Nice value Fork Inheritance (CPUS=1)
=====================================================================
[EXPECT] P and C should interleave evenly (e.g., PPCPCPCP...).

PPCPCPCPCPCPCPCPCPCPCPCPCPCPCPCPCPCPCPCPCPC
```

```
========================================================================
 TEST: 4. Nice value CPU Share (CPUS=1)
========================================================================
[EXPECT] H 16 times, L 1 time, H 16 times ...

LHLHHHHHHHHHHHHHHHLHHHHHHHHHHHHHHHHLHHHHHHHHHHHHHHHHLHHHHHHHHHHHLLLLL


========================================================================
 TEST: 5. Check for deadlocks in Multiprocessor environment
========================================================================
[EXPECT] No deadlocks occur & Process ends in A, B, C, D, E, F order or similar.

[a: -3] [b: -2] [c: -1] [d: 0] [e: 1] [f: 2]

abcdefabcdefabaacbaabaadcbaabaacbaabaaedcbaabaacbaabaadcbaabaacbaabaafedcbaabaacbaabaa
dcbaabaacbaabaaedcbaabaacbaabaadcbaabaacbaabaafedcbaabaacbaabaadcbaabaacbaabaaedcbaaba
acbaabaadcbaabaacbaabaafedcbaabAcbbdcbbcbbedcbbcbbdcbbcbbfedcbbcbbdcbbcbbedcbbcbbdcbbc
bbfedcbbcbbdcbbcbbedcbbcbbdcbbcbbfedcbbcBdccedccdccfedccdccedccdccfedccdccedccdccfedcc
dccedccdccfedccdccedccdccfedccdccedccdccfedccdCeddfeddeddfeddeddfeddeddfeddeddfeddeddf
eddeddfeddeddfeddeddfeddeddfeddeddfeddeddfeddeDfeefeefeefeefeefeefeefeefeefeefeefeefee
feefeefeefeefeefeefeefeefeefeefeefEfffffffffffffffffffffffffffffffffffffffffffffffF

================================================
 ALL TESTS COMPLETED.
================================================
PASS
PASS ALL TESTS
```

Troubleshooting

Lab 6 실습 때 했던 방식과 비슷한 접근을 사용했는데, Lab 6에서는 Queue 구조체를 사용하였고, 이번 프로젝트는 Red-Black Tree를 사용하다 보니 Red-Black Tree의 동작 방식을 이해하고 그에 맞게 프로세스를 삽입하고 프로세스의 정보를 가져오는 것이 필요했다.

초반에는 `rb_insert()` 함수를 사용하여 직접 RUNNABLE tree에 프로세스를 삽입하려 했는데, 이렇게 했을 때 `rb_insert()`를 실행할 때마다 `lock`에 대한 `acquire()` 함수와 `release()` 함수를 추가해줘야 한다. 그래서 직접 insert 함수를 따로 만들어서 이 문제를 해결하면서 동시에, `tree`에 프로세스(`proc` 구조체)를 직접 넣는 것이 조금 더 직관적인 것 같아서 프로세스를 매개변수로 받는 방식으로 만들었다.

비슷한 매커니즘으로 virtual runtime 최솟값을 구하는 방식도 따로 함수로 만들었다. 기존의 구현 계획은 Red-Black Tree의 가장 왼쪽 노드에 최솟값이 담겨 있기 때문에, 이를 찾아주는 `rbtree.c`에 구현된 `rb_first()` 함수를 이용하려 했다. 그러나 `rb_insert()` 함수와 마찬가지로 `lock`에 대한 기능에 더불어...

1. RUNNABLE 프로세스가 존재하는 경우
→ 명세대로 RUNNABLE 프로세스들 중 최소 virtual runtime으로 갱신

2. RUNNABLE 프로세스는 없지만, RUNNING 중인 프로세스가 하나 이상 존재하는 경우
→ RUNNING 프로세스들의 virtual runtime 중 최솟값으로 갱신

3. RUNNABLE도 RUNNING도 존재하지 않는 경우
→ virtual runtime을 0으로 설정

이 공지가 올라오면서 최소 virtual runtime 탐색 과정을 정리할 함수가 필요해졌다. 그래서 이 또한 `get_min_vruntime()`이라는 별도의 함수를 만들었다.

마지막으로 테스트 과정에서 CPU의 수를 1로 만들지 않으면(make qemu 실행) 원하는 대로 결과가 출력되지 않는 문제를 발견했다.

```
$ cfs
Starting CFS Test...
PIPID: 5, Nice: -2, Ticks: 11
D: 4, Nice: -3, Ticks: 11
PID: 6, Nice: -1, Ticks: 13
PID: 7, Nice: 0, Ticks: 14
PID: 8, Nice: 1, Ticks: 16
PID: 9, Nice: 2, Ticks: 17
CFS Test Completed.
```

멀티코어 환경에 맞게 구현했다면 nice값이 낮은 프로세스부터 차례대로 종료되는 게 맞지 않나?

알고 보니 nice값이 -2인 프로세스와 -3인 프로세스 간의 virtual runtime 차이가 크지 않아서 거의 동시에 끝나는 것처럼 보여서 발생한 문제였다. 그래서 `LOOP_COUNT`를 300,000,000(3억)에서 1,000,000,000(10억)으로 늘려주니 해결되었다. 추가로 멀티코어 환경에서의 문제가 발생한다면 이러한 스케줄링의 출력 순서가 아닌, deadlock으로 인한 panic이 출력될 것이므로 스케줄러 구현에는 문제가 없음을 확인하였다.